

Tietokannat jatkokurssi -lopputehtävä

Olli Hirvonen

Github -linkki (<https://github.com/Meanie-Beanie/Tietokannat-jatkokurssi-2026>).

HackMD linkki (<https://hackmd.io/@OlliHirvonen/r1RQW0IAWe>).



Tehtävänanto

Harjoitustyö: Autovuokraamon tietokantajärjestelmä (MariaDB)

Tavoite: Suunnitella ja toteuttaa relaatiotietokanta, joka hyödyntää edistyneitä MariaDB:n ominaisuuksia: SQL/PSM-ohjelmointia, triggereitä, transaktioita, tapahtumia ja roolipohjaista pääsynhallintaa.

Työn kuvaus

Tehtävänäsi on luoda taustajärjestelmä "GoAuto"-autovuokraamolle. Järjestelmän tulee hallita asiakkaita, ajoneuvoja ja varauksia turvallisesti, estää päällekkäiset varaukset automaattisesti sekä pitää kirjaa huolloista.

Vaihe 1: Suunnittelu, relaatiot ja rajoittimet

Suunnittele ja luo tietokanta nimeltä GoAutoDB. Tietokannan on täytettävä vähintään 3. normaalimuodon (3NF) vaatimukset.

Luo seuraavat taulut oikeaoppisine viiteavaimineen (FOREIGN KEY):

Asiakkaat (asiakas_id, etunimi, sukunimi, sähköposti, ajokortin_numero)

Autot (auto_id, rekisterinumero, merkki, malli, päivähinta, tila [esim. 'vapaa', 'vuokralla', 'huollossa'])

Varaukset (varaus_id, asiakas_id, auto_id, varaus_alkaa, varaus_paattyy, kokonaishinta)

Loki (loki_id, toimenpide, aikaleima, kenen_tekema)

Määritä rajoittimet (CONSTRAINTS):

Sähköpostin on oltava yksilöllinen (UNIQUE).

Auton päivähinnan on oltava suurempi kuin nolla (CHECK).

Varauksen päättymispäivän on oltava sama tai myöhäisempi kuin alkamispäivä (CHECK).

Vaihe 2: Tietokantaohjelmointi (SQL/PSM)

Käytä MariaDB:n SQL/PSM -kieltä luodaksesi tietokantaan sisäänrakennettua logiikkaa.

Tallennettu funktio (Stored Function): LaskeHinta

Luo funktio, joka ottaa parametreina auton ID:n, varauksen alkupäivän ja loppupäivän.

Funktio laskee ja palauttaa varauksen kokonaishinnan (päivien määrä $\times$ auton päivähinta).

Tallennettu proseduuuri (Stored Procedure): TeeVaraus

Proseduuri ottaa parametreina asiakkaan ID:n, auton ID:n, alkupäivän ja loppupäivän.

Transaktio: Proseduurin sisällä on käytettävä transaktiota (START TRANSACTION, COMMIT, ROLLBACK).

Logiikka: Tarkista ensin, onko auto vapaana kyseisenä ajankohtana. Jos on, lisää rivi Varaukset-tauluun ja päivitä auton tila Autot-taulussa. Jos auto on jo varattu, peruuta transaktio ja nosta virheilmoitus (SIGNAL SQLSTATE).

Vaihe 3: Triggerit ja automaatio

Lisää järjestelmään automaattisia tarkistuksia ja jäljitettävyyttä.

Audit-loki (AFTER UPDATE):

Luo triggeri Autot-tauluun. Aina kun auton tila muuttuu arvoon 'huollossa', triggerin tulee automaattisesti lisätä tietue Loki-tauluun (esim. "Auto [rekisterinumero] siirretty huoltoon").

Turvatarkistus (BEFORE INSERT):

Luo triggeri Varaukset-tauluun, joka estää päällekkäiset varaukset tietokantatasolla. Jos uusi varaus osuu aikavälille, jolloin auto on jo varattu, triggerin tulee pysäyttää operaatio ja palauttaa virhe.

Vaihe 4: Näkymät, käyttäjähallinta ja roolit

Näkymä (View): v_AktiivisetVaraukset

Luo näkymä, joka yhdistää tietoa tauluista: se näyttää asiakkaan nimen (muttei ajokortin numeroa), auton merkin, rekisterinumeron ja varauksen aikavälin. Näkymän tulee näyttää vain ne varaukset, jotka ovat tällä hetkellä käynnissä tai tulevaisuudessa.

Roolit (Roles) ja oikeudet:

Luo rooli asiakaspalvelu. Anna tälle roolille oikeus lukea (SELECT) ja lisätä (INSERT) tietoa Varaukset ja Asiakkaat -tauluihin, sekä oikeus ajaa proseduuri TeeVaraus.

Luo rooli mekaanikko. Anna tälle roolille oikeus päivittää (UPDATE) Autot-taulun tila-saraketta ja lukea v_AktiivisetVaraukset -näkymää.

Luo testikäyttäjät molemmille rooleille ja liitä roolit niihin (GRANT ROLE).

Vaihe 5: Suorituskyvyn optimointi ja tapahtumat

Suorituskyvyn optimointi (Indeksit ja EXPLAIN):

Kirjoita kysely, joka etsii tietyn asiakkaan kaikki varaukset. Aja kysely EXPLAIN-komennon läpi ja ota tuloste talteen.

Luo tarvittava indeksi (CREATE INDEX), joka nopeuttaa tätä kyselyä (esim. asiakas_id ja varaus_alkaa -sarakkeisiin).

Aja EXPLAIN uudelleen ja vertaa tuloksia dokumentaatiossasi.

Tapahtumat (Events):

Huom: Varmista ensin, että MariaDB:n tapahtuma-ajastin on päällä: SET GLOBAL event_scheduler = ON;

Luo tapahtuma PaivitaAutojenTilat, joka ajetaan automaattisesti kerran vuorokaudessa.

Tapahtuman tulee etsiä varaukset, jotka ovat päättyneet eilen, ja päivittää kyseisten autojen tila takaisin 'vapaa'-tilaan Autot-taulussa.

Palautusohjeet

Palauta työ yhtenä .zip-pakettina (tai GitHub-linkkinä), joka sisältää:

Tietokantaskripti (GoAuto_dump.sql tai vastaava): Täydellinen DDL- ja DML-skripti, jonka ajamalla tietokanta, rakenteet, funktiot, proseduurit, triggerit, eventit ja esimerkkidata (vähintään 5 riviä per taulu) luodaan onnistuneesti puhtaalle MariaDB-palvelimelle.

Dokumentaatio (Raportti.pdf):

Lyhyt kuvaus tietokannan rakenteesta (ER-kaavio on plussaa).

Todisteet indeksien toimivuudesta (kuvakaappaukset tai tekstitulosteet EXPLAIN-komennoista ennen ja jälkeen).

Lyhyt perustelu valitsemillesi rooleille ja niiden oikeuksille.

Kirjoita loppurefleksio/palaute opintojaksosta: mikä oli toimivaa ja mitä kannattaa vielä kehittää seuraavaan toteutukseen? Mikä aihe oli mielenkiintoista ja mikä ehkä meni hieman ohi tai ei ollut niin kiinnostavaa? Oliko toteutustapa juuri sinulle sopiva? Mitä terveisiä haluaisit lähettää opettajalle?

Projektin aloitus

Tietokantarakenne

Vaihe 1:n mukaisesti rakennamme seuraavat taulut:

Asiakkaat (asiakas_id, etunimi, sukunimi, sahoposti, ajokortin_numero)

Autot (auto_id, rekisterinnumero, merkki, malli, paivahinta, tila [esim. 'vapa

Varaukset (varaus_id, asiakas_id, auto_id, varaus_alkaa, varaus_paattyy, koko

Loki (loki_id, toimenpide, aikaleima, kenen_tekema)

-- Määritä rajoittimet (CONSTRAINTS):

Sähköpostin on oltava yksilöllinen (UNIQUE).

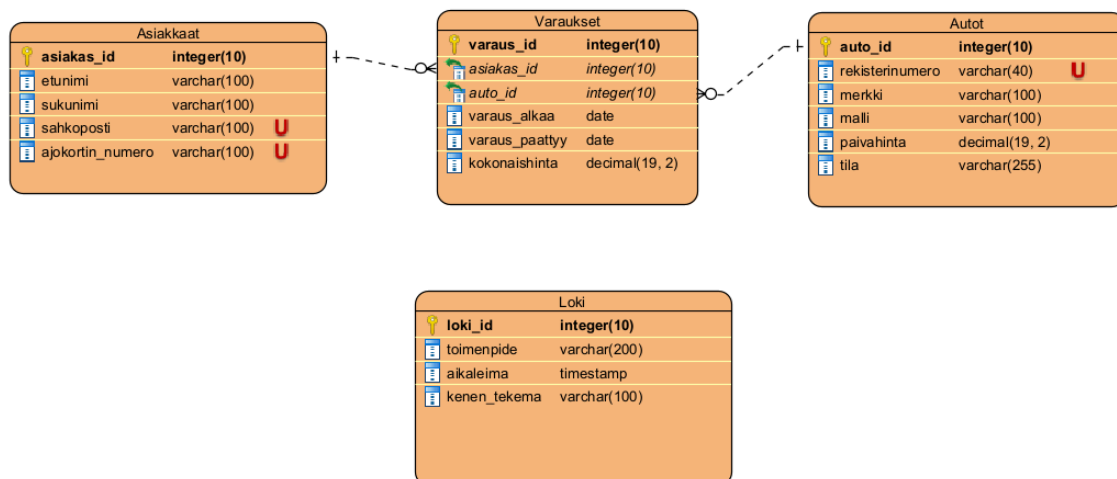
Auton päivähinnan on oltava suurempi kuin nolla (CHECK).

Varauksen päättymispäivän on oltava sama tai myöhäisempi kuin alkamispäivä (C

Rakennan näistä fyysisen ERD -kaavion Visual Paradigm -ohjelmalla

(<https://www.visual-paradigm.com/>), jonka perusteella rakennan ratkaisun. Kuva siitä löytyy tämän osion lopusta sekä itse tiedosto löytyy myös versionhallinnasta seuraavasta repositoriosta: Tietokannat jatkokurssi 2026

(<https://github.com/Meanie-Beanie/Tietokannat-jatkokurssi-2026>). Tästä myös alla kuva:



Huomioita ja muita mieleentulevia asioita:

1. ajokortin_numero ja rekisterinnumero eivät ole uniikkeja.

Tulen lisäämään tämän, vaikka tehtävänannossa sitä ei pyydetä, koska ei ole henkilöitä, joilla on jaettu ajokortti tai ajoneuvoja, joissa useassa on sama rekisterinnumero.

2. Tila -arvo

Tila -arvo on tehtävänannossa kuvailtu Enum -tyyppisenä, mutta jos olisin suunnitellut tämän itse, niin olisin luonut erillisen 'Tila' -taulun, jonka kautta tieto olisi liitetty 'Varaukset' -tauluun.

Syy tähän on kohtalaisen yksinkertainen: se on mielestäni selkeämpi sekä lisäykset ja muutokset on helpompi luoda.

4. Loki -taulun kenen_tekema -kolumni

Itseasiassa tässä tekemisen aikana olen uudestaan miettinyt tätä asemaa, koska alunperin olisin halunnut liittää käyttäjän FK:lla VARCHAR -tyyppisen arvon sijaan.

Mutta netissä on paljon keskustelua aiheesta ja tästä aiheutuu muutama ongelma:

1. Väite: Käyttäjän poistettaessa se poistaisi myös käyttäjän logitiedot.
Totuus: Voimme estää tämän hyödyntämällä CASCADE DELETE RESTRICT, joka estäisi käyttäjän poistamisen, jos sillä on olemassa logitiedostoja.
2. Väite: Logi -taulu halutaan olevan nopeasti tietoa haettava sekä kirjoittava taulu ja Foreign Key olisi hidastava tekijä. Totuus: Tämä aiheuttaa pientä performanssin hidastusta, mutta käytännössä sen vaikutus olisi aika pieni hyötyyn nähden.
3. Totuus: Liitettynä järjestelmätauluun, kuten Mysql.Users, tekee tietokannan siirtämisestä vaikean, jos se pitää siirtää eri versioon Mysql:stä tai esim. täysin eri SQL -moottoriin, kuten SQL Serveriin tai vaikka Oraclen DB:hen. Totuus:

Tämä olisi vähän helpompi ratkaisu, jos tällä olisi erillinen käyttäjäohjelmisto kirjautumisjärjestelmä. Voisin hyödyntää Käyttäjä -taulua ja yhdistäisin sen lokiin.

Mutta en liitä sitä mitenkään vaan luon loki -taulun hyödyntämään VARCHAR -arvoa, jolle vaan haetaan sen hetkisen käyttäjän nimi ja se syötetään string -arvolla kolumniin.

5. VARCHAR -arvojen koot

Yleensä aina, kun olen suunnitellut tietokantoja, olen antanut reilummin pelivaraa tietojen tallennusta varten.

Maksimiarvoja ei koskaan annetta, ellei siihen ole hyvää syytä ja en koskaan myöskään halua antaa sille minimiarvoa, koska silloin pienimmätkin muutokset saattavat rikkoa toimintaa.

Onko maailmassa oikeasti yli 50 kirjainta olevaa etunimeä? Ehkä muutama kappale koko maailmassa tai ei ollenkaan, mutta ainakin varmistamme, että kyseisen henkilön kohdalla järjestelmämme ei rikkoudu.

Ensimmäinen skripti (Perusrakenne, ei tarkistuksia)

```
-- Asiakkaat (asiakas_id, etunimi, sukunimi, sähköposti, ajokortin_numero)

Create Table Asiakkaat (
  asiakas_id int AUTO_INCREMENT PRIMARY KEY,
  etunimi varchar(100) NOT NULL,
  sukunimi varchar (100) NOT NULL,
  sähköposti varchar(100) NOT NULL UNIQUE,
  ajokortin_numero varchar(50) UNIQUE
);

-- Autot (auto_id, rekisterinnumero, merkki, malli, päivähinta, tila [esim. 'v

Create Table Autot (
  auto_id int AUTO_INCREMENT PRIMARY KEY,
  rekisterinnumero VARCHAR(20) NOT NULL UNIQUE,
  merkki VARCHAR(50) NOT NULL,
  malli VARCHAR(50) NOT NULL,
  päivähinta DECIMAL(8,2) NOT NULL,
  tila ENUM('vapaa', 'vuokralla', 'huollossa') NOT NULL DEFAULT 'vapaa'
);

-- Varaukset (varaus_id, asiakas_id, auto_id, varaus_alkaa, varaus_paattyy, k

Create Table Varaukset (
  varaus_id int AUTO_INCREMENT PRIMARY KEY,
  varaus_alkaa DATETIME DEFAULT CURRENT_TIMESTAMP,
  varaus_paattyy DATETIME,
  kokonaishinta decimal(8,2) NOT NULL,
  asiakas_id int NOT NULL,
  auto_id int NOT NULL,

  CONSTRAINT fk_asiakas_id
  FOREIGN KEY (asiakas_id)
  REFERENCES Asiakkaat(asiakas_id),

  CONSTRAINT fk_auto_id
  FOREIGN KEY (auto_id)
  REFERENCES Autot(auto_id)
);

-- Loki (loki_id, toimenpide, aikaleima, kenen tekemä)

Create Table Loki(
  loki_id int AUTO_INCREMENT PRIMARY KEY,
  toimenpide VARCHAR(40) NOT NULL,
  aikaleima DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
kenen_tekema VARCHAR(100) NOT NULL  
);
```

Skriptissä saattaa olla virheitä, etenkin mitä tulee avainsanoihin. Ne vaihtelevat hieman eri SQL moottorien välillä, joten saatan suoritus vaiheessa huomata, että se onkin eri moottorin avainsana.

Ylläolevaan skriptistä puuttuu vielä tarkistukset ja muut vaatimukset, mutta loin alustavan rakenteen näin alkuun.

Jos vertaamme aikaisemmin antamaani kuvaan, niin siellä muutamat asiat muuttuivat:

1. Muutama NULL -tarkistus lisää.
2. Desimaalilukujen pienennys 19,2 -> 8,2, koska yhdeksääntoista lukuun saadaan aika monta numeroa laitettua, niin pienensin sen huomattavasti alemmaksi.
3. ENUM -arvolla on DEFAULT arvona 'vapaa' sekä DATETIME ottaa luontihetken ajan useimmissa tapauksissa, ellei sitä erikseen anneta.

Toinen skripti (Lisärajoituksineen)

- 1) Sähköpostin on oltava yksilöllinen (UNIQUE).
- 2) Auton päivähinnan on oltava suurempi kuin nolla (CHECK).
- 3) Varauksen päättymispäivän on oltava sama tai myöhäisempi kuin alkamispäivä



Sähköposti (ja muutamat muut kolumnit) ovat jo uniikkeja.

En ole itseasiassa koskaan tehnyt CHECK -rajoituksia aikaisemmin. Olen yleensä ohjelman kautta rajoittanut tiedon siirtoa, en tietokannan sisäisesti.

Onneksi tämä (https://www.w3schools.com/sql/sql_check.asp) ei itseasiassa ole hyvin monimutkainen. Alla on päivitetty versio:

```
CREATE DATABASE IF NOT EXISTS GoAuto;

USE GoAuto;

-- Asiakkaat (asiakas_id, etunimi, sukunimi, sähköposti, ajokortin_numero)

Create Table Asiakkaat (
asiakas_id int AUTO_INCREMENT PRIMARY KEY,
etunimi varchar(100) NOT NULL,
sukunimi varchar (100) NOT NULL,
sähköposti varchar(100) NOT NULL UNIQUE,
ajokortin_numero varchar(50) UNIQUE
);

-- Autot (auto_id, rekisterinnumero, merkki, malli, päivähinta, tila [esim. 'v
-- Auton päivähinnan on oltava suurempi kuin nolla (CHECK).

Create Table Autot (
auto_id int AUTO_INCREMENT PRIMARY KEY,
rekisterinnumero VARCHAR(20) NOT NULL UNIQUE,
merkki VARCHAR(50) NOT NULL,
malli VARCHAR(50) NOT NULL,
päivähinta DECIMAL(8,2) NOT NULL CHECK (päivähinta > 0),
tila ENUM('vapaa', 'vuokralla', 'huollossa') NOT NULL DEFAULT 'vapaa'
);

-- Varaukset (varaus_id, asiakas_id, auto_id, varaus_alkaa, varaus_paattyy, k
-- Varauksen päättymispäivän on oltava sama tai myöhäisempi kuin alkamispäivä

Create Table Varaukset (
varaus_id int AUTO_INCREMENT PRIMARY KEY,
varaus_alkaa DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
varaus_paattyy DATETIME NOT NULL,
kokonaishinta decimal(8,2) NOT NULL,
asiakas_id int NOT NULL,
auto_id int NOT NULL,

CONSTRAINT chk_varaus_aika
CHECK (varaus_paattyy >= varaus_alkaa),

CONSTRAINT fk_asiakas_id
FOREIGN KEY (asiakas_id)
REFERENCES Asiakkaat(asiakas_id),

CONSTRAINT fk_auto_id
FOREIGN KEY (auto_id)
REFERENCES Autot(auto_id)
);

-- Loki (loki_id, toimenpide, aikaleima, kenen tekemä)
```

```
Create Table Loki(  
  loki_id int AUTO_INCREMENT PRIMARY KEY,  
  toimenpide VARCHAR(40) NOT NULL,  
  aikaleima DATETIME DEFAULT CURRENT_TIMESTAMP,  
  kenen_tekema VARCHAR(100) NOT NULL  
);
```

Vähän oli kökköä, että W3schools sivuilla () usean rajoitteen esimerkki oli demonstroitu sen nimeämis -osion alla eikä omassa osiossaan tai liitettynä ensimmäiseen demonstraatioon.

Itseasiassa kokonaishinnassa voisi olla CHECK -tarkistus, että se ei voisi olla esim. negatiivinen?

Itseasiassa CASCADE -käyttäytymistä ei ole määritetty ja nopeasti netistä lukemalla se pitäisi olla RESTRICT. Tämä on ihan tarpeeksi hyvä, koska se estää massadatan tuhoutumista ja sinun pitää tietää tarkalleen, mitä poistat ja mistä. Se on myös rasittavaa, koska jos sinun pitää poistaa kaikki tieto siitä, sinun pitää tietää tarkalleen missä siihen viitataan.

Päivitys: lisäsin Autot tilaan myös 'varattu', koska se saattaisi olla hyödyllinen tieto, jos auto on varattu, mutta se ei ole vielä vuokralla.

Vaihe 2: Tietokantaohjelmointi (SQL/PSM)

Tallennettu funktio (Stored Function): LaskeHinta

"Luo funktio, joka ottaa parametreina auton ID:n, varauksen alkupäivän ja loppupäivän.

Funktio laskee ja palauttaa varauksen kokonaishinnan (päivien määrä × auton päivähinta)."

<https://www.tutorialspoint.com/mysql/mysql-stored-functions.htm>

(<https://www.tutorialspoint.com/mysql/mysql-stored-functions.htm>)

Esim:

```
DELIMITER $$
CREATE OR REPLACE FUNCTION LaskeHinta(IN p_auto_id INT, IN p_varaus_alkaa DAT
RETURNS DECIMAL(8,2)
DETERMINISTIC
BEGIN
    DECLARE kokonaishinta DECIMAL(8,2);
    DECLARE paivien_maara int;
    DECLARE paivahinta DECIMAL(8,2);

    SET paivien_maara = DATEDIFF(p_varaus_paattyy, p_varaus_alkaa);
    SELECT auto.paivahinta INTO paivahinta
    FROM autot auto
    WHERE auto.auto_id = p_auto_id;

    SET kokonaishinta = paivahinta * paivien_maara;

    RETURN kokonaishinta;
END$$
DELIMITER ;
```

Funktion kutsuesimerkki:

```
-- Auto -olio {auto_id: 1, rekisterinnumero: 'ABC-123', merkki: 'Toyota', mall

SELECT LaskeHinta(1, '2026-05-1', '2026-05-6') AS kokonaishinta;

-- palauttaa arvon: 225, joka on 45 * 5.
```

Itseasiassa hauskasti huomasin vasta nyt, että periaatteessa yksi päivä jää pois eli jos asiakas hakee auton 1. päivä, niin sehän on periaatteessa 6 päivän verran hänen käytössään. Mutta vähän tätä asiaa tutkiessani tulin siihen päätökseen, että vuokraustavat ja maksut vaihtelevat aika paljon eri yrityksien välillä, että annan nyt tämän olla vain tämän autoliikkeen tapa maksuttaa.

Tallennettu proseduri (Stored Procedure): TeeVaraus

"Proseduuri ottaa parametreina asiakkaan ID:n, auton ID:n, alkupäivän ja loppupäivän.

Transaktio: Proseduurin sisällä on käytettävä transaktiota (START TRANSACTION, COMMIT, ROLLBACK).

Logiikka: Tarkista ensin, onko auto vapaana kyseisenä ajankohtana. Jos on, lisää rivi Varaukset-tauluun ja päivitä auton tila Autot-taulussa. Jos auto on jo varattu, peruuta transaktio ja nosta virheilmoitus (SQLSTATE)."

```
1  DELIMITER $$
2  CREATE OR REPLACE PROCEDURE TeeVaraus(IN p_asiakas_id INT, IN p_auto_id
3  BEGIN
4  DECLARE varauksien_maara INT;
5
6  START TRANSACTION;
7
8  SELECT COUNT(*) INTO varauksien_maara FROM Varaukset v
9  WHERE v.auto_id = p_auto_id
10 AND v.varaus_alkaa < p_varaus_paattyy
11 AND v.varaus_paattyy > p_varaus_alkaa
12 AND v.auto.tila = 'vapaa';
13
14 IF varauksien_maara = 0 THEN
15
16     INSERT INTO Varaukset(
17     varaus_alkaa,
18     varaus_paattyy,
19     kokonaishinta,
20     asiakas_id,
21     auto_id)
22     VALUES(
23     p_varaus_alkaa,
24     p_varaus_paattyy,
25     LaskeHinta(p_auto_id, p_varaus_alkaa, p_varaus_paattyy),
26     p_asiakas_id,
27     p_auto_id);
28
29     UPDATE Autot
30     SET Tila = 'varattu'
31     WHERE auto_id = p_auto_id;
32
33     COMMIT;
34
35 ELSE
36     ROLLBACK;
37     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto on varattuna v.
38 END IF;
39
40 END$$
41 DELIMITER ;
```

Täytyy sanoa, että transactioneita pitäisi käyttää useammin ihan kyselyitä tehdessä. Olisin säästynyt monelta päänvaivalta. Satuini nimittäin poistamaan yhden version tietokannasta tietoineen.

Mutta aivoni sulii tätä proseduuria miettiessä, koska päivien vertailu ja laskeminen aiheutti jo suurta tuskaa, mutta tämä on loogisesti yllättävän vaikea tilanne. Minulla oli kolme ongelmatilannetta mielessä, mutta olen jo unohtanut yhden niistä, mutta tässä on ne kaksi muuta:

1. Ongelmatilanne: Auton tilan päivitys

Aikaisemmin päivitin auton tilan 'vuokralla', mutta jos auto on varattu tulevaisuutta varten, sehän ei ole vuokralla vaan varattu.

Teen erillisen tapahtuman, joka tarkistaa päivittäin, että auto, joka on merkitty 'varattu', mutta aloituspäivä on jo ohitettu. Jos ei ole, niin se muuttaa sen vapaaksi.

Huom: Auton tila ei koskaan muutu parhaillaan 'varatuksi', koska se ei kuulu tehtävänantoon.

2. Ongelmatilanne: Toimintojen päällekkäisyys

Tiedän, että SQL:ssä pystyy lukitsemaan tauluja/kolumneja ja sellainen olisi varmaan hyvä tälläisessä tilanteessa, jossa usea käyttäjä voisi tehdä varauksia samanaikaisesti.

Kurssilla tästä ei ole tainnut olla mainintaa ja se saattaisi olla ihan mielenkiintoinen aihe tulevaisuuden kursseille.

Olen joutunut kyllä ihan tarpeeksi lukemaan dokumentaatiota ja oppimaan tätä projektia tehdessä, jonka takia en kyllä rupea tässä siihen perehtymään.

Vaihe 3: Triggerit ja automaatio

Huom. Käytin nimeämisessä seuraavia käytäntöjä:

`tr_{etukirjaimet_ajoituksesta}_{mitä_tekee}`

esim.

`tr_au_autot_huollossa_loki_lisays`

eli toisinsanoen:

trigger_after_update_huollossa_loki_lisays

Audit-loki (AFTER UPDATE)

"Luo triggeri Autot-tauluun. Aina kun auton tila muuttuu arvoon 'huollossa', triggerin tulee automaattisesti lisätä tietue Loki-tauluun (esim. "Auto [rekisterinumero] siirretty huoltoon")."

```
DELIMITER $$
CREATE TRIGGER tr_au_autot_huollossa_loki_lisays
AFTER UPDATE ON Autot
FOR EACH ROW
BEGIN
    IF NEW.tila = 'huollossa' THEN
        INSERT INTO Loki(toimenpide, aikaleima, kenen_tekema)
        VALUES(CONCAT('Auto ', OLD.rekisterinumero, ' siirretty huolt
    END IF;
END$$
DELIMITER ;
```



Turvataarkistus (BEFORE INSERT):

"Luo triggeri Varaukset-tauluun, joka estää päällekkäiset varaukset tietokantatasolla. Jos uusi varaus osuu aikavälille, jolloin auto on jo varattu, triggerin tulee pysäyttää operaatio ja palauttaa virhe."

```
DELIMITER $$
CREATE TRIGGER tr_bi_tarkista_onko_auto_varattu
BEFORE INSERT ON Varaukset
FOR EACH ROW
BEGIN
DECLARE varauksien_maara INT;

SELECT COUNT(*) INTO varauksien_maara FROM Varaukset v
WHERE v.auto_id = NEW.auto_id
AND v.varaus_alkaa < NEW.varaus_paattyy
AND v.varaus_paattyy > NEW.varaus_alkaa;

IF varauksien_maara > 0 THEN
    IGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto on varattuna valittuna aj
END IF;

END$$
DELIMITER ;
```



Logiikka tarkistukseen on otettu suoraan aikaisemmasta proseduuristamme.

Vaihe 4: Näkymät, käyttäjähallinta ja roolit

Näkymä (View): v_AktiivisetVaraukset

"Luo näkymä, joka yhdistää tietoa tauluista: se näyttää asiakkaan nimen (muttei ajokortin numeroa), auton merkin, rekisterinumeron ja varauksen aikavälin. Näkymän tulee näyttää vain ne varaukset, jotka ovat tällä hetkellä käynnissä tai tulevaisuudessa."

```
CREATE VIEW v_AktiivisetVaraukset AS
SELECT CONCAT(asiakas.etunimi, " ", asiakas.sukunimi),
a.merkki, a.rekisterinumero, v.varaus_alkaa, v.varaus_paattyy FROM varaukset
JOIN autot a ON a.auto_id = v.auto_id
JOIN asiakkaat asiakas ON asiakas.asiakas_id = v.asiakas_id
WHERE v.varaus_paattyy > NOW();
```



Itseasiassa yllätyin miten helppoa näkymöitä oli luoda ja käyttää. En ole koskaan tehnyt tätä aikaisemmin ja täytyy kyllä sanoa, että tämä on todella kätsy juttu.

```
-- Kutsumme sitä seuraavalla tavalla:
SELECT * from v_aktiivisetvaraukset;
```

Asiakaspalvelu -rooli

"Luo rooli asiakaspalvelu. Anna tälle roolille oikeus lukea (SELECT) ja lisätä (INSERT) tietoa Varaukset ja Asiakkaat -tauluihin, sekä oikeus ajaa proseduuuri TeeVaraus."

```
CREATE ROLE IF NOT EXISTS 'asiakaspalvelu';
GRANT SELECT, INSERT ON GoAutoDB.Varaukset TO 'asiakaspalvelu';
GRANT SELECT, INSERT ON GoAutoDB.Asiakkaat TO 'asiakaspalvelu';
GRANT EXECUTE ON PROCEDURE GoAutoDB.TeeVaraus TO 'asiakaspalvelu';
```

En ole juurikaan tunnuksiakaan luonut aikaisemmin, joten en tiennyt, miten eri tavoilla niiden oikeuksia voidaan rajata. Kiva!

Mekaanikko -rooli

Luo rooli mekaanikko. Anna tälle roolille oikeus päivittää (UPDATE) Autot-taulun tila-saraketta ja lukea v_AktiivisetVaraukset -näkymää.

```
CREATE ROLE IF NOT EXISTS 'mekaanikko';
GRANT UPDATE(tila) ON GoAutoDB.Autot TO 'mekaanikko';
GRANT SELECT ON GoAutoDB.v_aktiivisetvaraukset TO 'mekaanikko';
```

Ensimmäiseksi minulla oli laitettu UPDATE -oikeudet koko tauluun, mutta tämä tuli todella suurena yllätyksenä, että nämä oikeudet voidaan antaa KOLUMNI kohtaisesti. Tiesin, että ne voidaan rajata tauluttain ja esim. luku-/kirjoitusoikeuksilla, mutta en tiennyt, että ne voidaan myös rajata noin tarkasti.

Vaikein asia minun oli kirjoittaa sana 'mekaanikko' oikein. Jouduin sen varmaan korjamaan itse SQL skriptiin noin 5 kertaa.

Tunnuksien luonti ja oikeuksien antaminen

```
CREATE USER IF NOT EXISTS 'asiakaspalvelu'@'localhost' IDENTIFIED BY 'PASSWORD';
GRANT 'asiakaspalvelu' TO 'asiakaspalvelu'@'localhost';
SET DEFAULT ROLE 'asiakaspalvelu' FOR 'asiakaspalvelu'@'localhost';

CREATE USER IF NOT EXISTS 'mekaanikko'@'localhost' IDENTIFIED BY 'PASSWORD';
GRANT 'mekaanikko' TO 'mekaanikko'@'localhost';
SET DEFAULT ROLE 'mekaanikko' FOR 'mekaanikko'@'localhost';
```



Tämä kirjoitus vaihe on ollut hyödyllinen, koska tajusin tätä laittaessa, että olin vahingossa laittanut tuon sähköposti -muotoon ensimmäiseksi eli

'mekaanikko@localhost.com', vaikka tarkoituksena se oli laittaa 'localhost', koska nämähän ovat kaksi eri asiaa.

Lisäyksenä laitoin myös SET DEFAULT ROLE, joka varmistaa, että käyttäjän kirjaututtua hänellä on ne oikeudet. JOS tätä ei olisi, käyttäjä joutuisi manuaalisesti "hyväksymään oikeuksien käytöt".

Muut potentiaaliset roolit

En ole luonut skriptiin seuraavia rooleja, vaan ne on ihan vain puhdasta mietiskelyä.

1. Ylläpitäjä, jolla on kaikki oikeudet.
2. Varausteknikko, jolla on oikeudet tehdä suurempia muokkauksia Varaukset -tauluun (lisäys, poisto, päivitys, proseduurit jne.). Jos tulee ongelmatilanne asiakaspalvelun kanssa, teknikko hoitaisi.
3. Raportointi. Lukuoikeudet tauluihin, joista lukuoikeus esim. Asiakkaat - tauluun olisi rajattu (esim. ei näkisi sosiaalitunnusta tai ajokortin numeroa).

Loppusanat

Itseasiassa tämä osio oli yllättävän vaikea. Se on ns. yksinkertainen asia, mutta siellä on todella monia eri asioita, jotka voivat mennä pieleen ja pitää ottaa huomioon. Alla on muutamia lähteitä, mitä hyödynsin mysql dokumentaation lisäksi (joka oli tähän asiaan liittyen mielestäni sekava).

<https://stackoverflow.com/questions/10089308/mysql-stored-procedure-permissions> (<https://stackoverflow.com/questions/10089308/mysql-stored-procedure-permissions>).

<https://stackoverflow.com/questions/3108656/grant-a-user-permission-to-only-view-a-mysql-view-and-nothing-else> (<https://stackoverflow.com/questions/3108656/grant-a-user-permission-to-only-view-a-mysql-view-and-nothing-else>).

<https://stackoverflow.com/questions/16674715/granting-permission-to-individual-fields-in-mysql> (<https://stackoverflow.com/questions/16674715/granting-permission-to-individual-fields-in-mysql>).

Vaihe 5: Suorituskyvyn optimointi ja tapahtumat

Suorituskyvyn optimointi (Indeksit ja EXPLAIN):

"Kirjoita kysely, joka etsii tietyn asiakkaan kaikki varaukset. Aja kysely EXPLAIN-komennon läpi ja ota tuloste talteen.

Luo tarvittava indeksi (CREATE INDEX), joka nopeuttaa tätä kyselyä (esim. asiakas_id ja varaus_alkaa -sarakkeisiin).

Aja EXPLAIN uudelleen ja vertaa tuloksia dokumentaatiossasi."

```
EXPLAIN SELECT v.asiakas_id, v.auto_id, v.varaus_alkaa, v.varaus_paattyy FROM
JOIN Asiakkaat a ON v.asiakas_id = a.asiakas_id
WHERE v.asiakas_id = 1
```

Tuottaa seuraavanlaisen kyselyn.



The screenshot shows a database client interface. At the top, there is a SQL query editor with the following text:

```
1 EXPLAIN SELECT v.asiakas_id, v.auto_id, v.varaus_alkaa, v.varaus_paattyy FROM Varaukset v
2 JOIN Asiakkaat a ON v.asiakas_id = a.asiakas_id;
3
```

Below the editor, there is a table titled "Varaukset (2r x 10q)". The table has 10 columns: #, 1 id, 2 select_type, 3 table, 4 type, 5 possible_keys, 6 key, 7 key_len, 8 ref, 9 rows, and 10 Extra. The table contains two rows of data:

#	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	v	ALL	fk_asiakas_id	(NULL)	(NULL)	(NULL)	1	
2	1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	goauto.v.asiakas_id	1	

```
CREATE INDEX idx_varaukset_asiakas_varaus_alkaa ON Varaukset (asiakas_id, var
```

Luomme ylläolevalla kyselyllä indeksin, jonka jälkeen suoritamme EXPLAIN:in uudelleen.

```
EXPLAIN SELECT v.asiakas_id, v.auto_id, v.varaus_alkaa, v.varaus_paattyy FROM
JOIN Asiakkaat a ON v.asiakas_id = a.asiakas_id
WHERE v.asiakas_id = 1;
```

```

1 EXPLAIN SELECT v.asiakas_id, v.auto_id, v.varaus_alkaa, v.varaus_paattyy FROM Varaukset v
2 JOIN Asiakkaat a ON v.asiakas_id = a.asiakas_id;
3
4

```

#	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	v	ALL	idx_varaukset_asiakas_varaus_alkaa	(NULL)	(NULL)	(NULL)	1	
2	1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	goauto.v.asiakas_id	1	

Kysely on vähän huono esimerkki, koska emme hyödynnä varaus_alkaa -parametriä, mutta EXPLAIN komennolla huomaamme kummiskin eron; possible_keys sarakkeen alta löytyy nyt luomamme indeksiavain.

Lisäksi indeksointi on sellainen asia, joka nopeuttaa prosessia vain tiedon määrän kasvaessa. Pienillä datamäärillä se on itseasiassa hidastava tekijä, koska se hidastaa päivitys- ,lisäys- sekä poisto -operaatioita. Tästä mainitaan luentomateriaaleissa sekä tällä videolla (<https://www.youtube.com/watch?v=t0grczCICMk>).

Tapahtumat (Events)

Huom: Varmista ensin, että MariaDB:n tapahtuma-ajastin on päällä: SET GLOBAL event_scheduler = ON;

```
1 | SET GLOBAL event_scheduler = ON;
```

Yleisiä mietintöjä

1. Mitä tapahtuu, jos palvelin on esim. sammunut tapahtuman laukaisuaikana? Käynnistyykö se palvelimen käynnistymisen yhteydessä?

Minulle tuli tämä kysymys mieleen, kun halusin testata tapahtuman toimintaa.

Vastaus on, että tapahtumaa ei suoriteta ja sitä ei myöskään suoriteta jälkikäteen. Se vain suoritetaan sen suoritushetkellä. Toisin sanoen, jos keskiyön lähellä tulee katkos, on mahdollista, että tapahtumamme ei suoriudu ja autot eivät vapaannu.

2. Tapahtuma -skriptini ei sisältänyt aloitusaikaa.

Ensimmäisen kysymyksen perusteella minulle tuli seuraava asia mieleen, koska minulla ei ollut merkitty aloitusaikaa tapahtuman luontiskriptiin, joka on täysin validia SQL:llä.

Jos aloitusaikaa ei ole merkitty, tietokantamoottori luo sille aloitusajan sen perusteella, milloin tapahtuma on luotu, jonka perusteella se tulee suorittamaan tapahtumia tulevaisuudessa.

Esim:

Jos tapahtuma on luotu kello 11:30 ja se suoritetaan joka päivä, tapahtuma suoritetaan joka päivä aina kello 11:30.

Tämän tiedon perusteella, lisäsin tapahtumiini kurssimateriaalissa mainitun aloitusajan, joka suorittaa tapahtuman kello 23:59 joka päivä riippumatta tapahtuman luontiajankohdasta.

PaivitaAutojenTilat

"Luo tapahtuma PaivitaAutojenTilat, joka ajetaan automaattisesti kerran vuorokaudessa.

Tapahtuman tulee etsiä varaukset, jotka ovat päättyneet eilen, ja päivittää kyseisten autojen tila takaisin 'vapaa'-tilaan Autot-taulussa."

```
DELIMITER $$
CREATE EVENT PaivitaAutojenTilat
ON SCHEDULE
    EVERY 1 DAY
    STARTS CURRENT_DATE + INTERVAL 23 HOUR + INTERVAL 59 MINUTE
    COMMENT 'Päättyneiden varauksien autot on laitettu vapaiksi.'
DO
    BEGIN
        UPDATE Autot a
        JOIN Varaukset v ON v.auto_id = a.auto_id
        SET a.tila = 'vapaa'
        WHERE v.varaus_paattyy <= NOW() - INTERVAL 1 DAY
        AND a.tila = 'vuokralla';
    END $$
DELIMITER ;
```

Tämä tapahtuma kerkesi muuttumaan useaan otteeseen ja siinä on vieläkin loogisia ongelmia. Se tekee kummiskin, mitä tehtävänanto vaatii.

Yksi iso ongelma siinä on, että jos auto on varattu toiselle henkilölle varauksen päädyttyä tai se tullaan varamaan myöhemmin, tämä tulee virheellisesti laittamaan sen vapaaksi.

VapautaKayttamatonVaraus

Koska loin uuden tilan nimeltään 'varattu', kun auto on varattu, mutta ei varaus ei ole vielä alkanut, loin myös toisen tapahtuman, joka purkaa varatun tilan, jos autoa ei esim. haettu.

```
DELIMITER $$
CREATE EVENT VapautaKayttamatonVaraus
ON SCHEDULE
    EVERY 1 DAY
    STARTS CURRENT_DATE + INTERVAL 23 HOUR + INTERVAL 59 MINUTE
    COMMENT 'Käyttämättömät varaukset ovat vapautettu.'
DO
    BEGIN
        UPDATE Autot a
        JOIN Varaukset v ON v.auto_id = a.auto_id
        SET a.tila = 'vapaa'
        WHERE v.varaus_alkaa < NOW()
        AND a.tila = 'varattu';
    END $$
DELIMITER ;
```

Tietokantaskripti (Löytyy myös täältä githubista)

(<https://github.com/Meanie-Beanie/Tietokannat-jatkokurssi-2026>)

```
DROP DATABASE IF EXISTS GoAutoDB;  
CREATE DATABASE IF NOT EXISTS GoAutoDB;
```

```
USE GoAutoDB;
```

```
-- Asiakkaat (asiakas_id, etunimi, sukunimi, sahkoposti, ajokortin_numero)  
Create Table Asiakkaat (  
  asiakas_id int AUTO_INCREMENT PRIMARY KEY,  
  etunimi varchar(100) NOT NULL,  
  sukunimi varchar (100) NOT NULL,  
  sahkoposti varchar(100) NOT NULL UNIQUE,  
  ajokortin_numero varchar(50) UNIQUE  
);
```

```
-- Autot (auto_id, rekisterinnumero, merkki, malli, paivahinta, tila [esim. 'v  
-- Auton päivähinnan on oltava suurempi kuin nolla (CHECK).  
Create Table Autot (  
  auto_id int AUTO_INCREMENT PRIMARY KEY,  
  rekisterinnumero VARCHAR(20) NOT NULL UNIQUE,  
  merkki VARCHAR(50) NOT NULL,  
  malli VARCHAR(50) NOT NULL,  
  paivahinta DECIMAL(8,2) NOT NULL CHECK (paivahinta > 0),  
  tila ENUM('vapaa', 'varattu' , 'vuokralla', 'huollossa') NOT NULL DEFAULT 'va  
);
```

```
-- Varaukset (varaus_id, asiakas_id, auto_id, varaus_alkaa, varaus_paattyy, k  
-- Varauksen päättymispäivän on oltava sama tai myöhäisempi kuin alkamispäivä  
Create Table Varaukset (  
  varaus_id int AUTO_INCREMENT PRIMARY KEY,  
  varaus_alkaa DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  varaus_paattyy DATETIME NOT NULL,  
  kokonaishinta decimal(8,2) NOT NULL,  
  asiakas_id int NOT NULL,  
  auto_id int NOT NULL,
```

```
CONSTRAINT chk_varaus_aika  
CHECK (varaus_paattyy >= varaus_alkaa),
```

```
CONSTRAINT fk_asiakas_id  
FOREIGN KEY (asiakas_id)  
REFERENCES Asiakkaat(asiakas_id),
```

```
CONSTRAINT fk_auto_id  
FOREIGN KEY (auto_id)  
REFERENCES Autot(auto_id)  
);
```

```
-- Loki (loki_id, toimenpide, aikaleima, kenen tekemä)
```

```
Create Table Loki(  
  loki_id int AUTO_INCREMENT PRIMARY KEY,  
  toimenpide VARCHAR(40) NOT NULL,  
  aikaleima DATETIME DEFAULT CURRENT_TIMESTAMP,  
  kenen_tekema VARCHAR(100) NOT NULL  
);  
  
-- Asiakkaat -data  
INSERT INTO Asiakkaat (etunimi,sukunimi,sahkoposti,ajokortin_numero)  
VALUES  
('Mikko', 'Virtanen', 'mikko.virtanen@email.fi', '123456'),  
('Sanna', 'Korhonen', 'sanna.korhonen@email.fi', '234567'),  
('Juhani', 'Mäkinen', 'juhani.makinen@email.fi', '345678'),  
('Laura', 'Niemi', 'laura.niemi@email.fi', '456789'),  
('Pekka', 'Hämäläinen', 'pekka.hamalainen@email.fi', '567890'),  
('Erika', 'Leinonen', 'erika.leinonen@email.fi', '678901');  
  
-- Autot -data  
INSERT INTO Autot (rekisterinumero,merkki, malli,paivahinta,tila)  
VALUES  
('ABC-123', 'Toyota', 'Corolla', 45.00, 'vapaa'),  
('DEF-456', 'Volkswagen', 'Golf', 55.00, 'vapaa'),  
('GHI-789', 'Ford', 'Focus', 50.00, 'vapaa'),  
('JKL-012', 'BMW', '3 Series', 95.00, 'vapaa'),  
('MNO-345', 'Skoda', 'Octavia', 48.00, 'huollossa'),  
('PQR-678', 'Volvo', 'V60', 80.00, 'vapaa');  
  
-- Varaukset  
INSERT INTO Varaukset (varaus_alkaa, varaus_paattyy, kokonaishinta, asiakas_i  
VALUES  
('2026-06-01', '2026-06-04', 135.00, 1, 1),  
('2026-06-05', '2026-06-10', 275.00, 2, 2),  
('2026-06-03', '2026-06-05', 100.00, 3, 3),  
('2026-06-07', '2026-06-11', 380.00, 4, 4),  
('2026-06-10', '2026-06-11', 45.00, 5, 1),  
('2026-06-15', '2026-06-22', 560.00, 6, 6);  
  
-- Loki -data (simuloitu)  
INSERT INTO Loki (toimenpide,aikaleima,kenen_tekema)  
VALUES  
('ABC-123 siirretty huoltoon.', '2025-10-12', 'mekaanikko'),  
('DEF-456 siirretty huoltoon.', '2025-10-12', 'mekaanikko'),  
('GHI-789 siirretty huoltoon.', '2025-10-12', 'mekaanikko'),  
('JKL-012 siirretty huoltoon.', '2025-10-12', 'mekaanikko'),  
('MNO-345 siirretty huoltoon.', '2025-10-12', 'mekaanikko'),  
('PQR-678 siirretty huoltoon.', '2025-10-12', 'mekaanikko');  
  
-- Luo funktio, joka ottaa parametreina auton ID:n, varauksen alkupäivän ja l  
-- Funktio laskee ja palauttaa varauksen kokonaishinnan (päivien määrä $time  
DELIMITER $$  
CREATE OR REPLACE FUNCTION LaskeHinta(IN p_auto_id INT, IN p_varaus_alkaa DAT  
RETURNS DECIMAL(8,2)  
DETERMINISTIC
```

```
BEGIN

    DECLARE kokonaishinta DECIMAL(8,2);
    DECLARE paivien_maara int;
    DECLARE paivahinta DECIMAL(8,2);

    SET paivien_maara = DATEDIFF(p_varaus_paattyy, p_varaus_alkaa);
    SELECT auto.paivahinta INTO paivahinta
    FROM autot auto
    WHERE auto.auto_id = p_auto_id;

    SET kokonaishinta = paivahinta * paivien_maara;

    RETURN kokonaishinta;
END$$
DELIMITER ;

-- Tallennettu proseduuuri (Stored Procedure): TeeVaraus
-- Proseduuuri ottaa parametreina asiakkaan ID:n, auton ID:n, alkupäivän ja lo
-- Transaktio: Proseduurin sisällä on käytettävä transaktiota (START TRANSACT
-- Logiikka: Tarkista ensin, onko auto vapaana kyseisenä ajankohtana.
-- Jos on, lisää rivi Varaukset-tiluun ja päivitä auton tila Autot-tilussa.
-- Jos auto on jo varattu, peruuta transaktio ja nosta virheilmoitus (SIGNAL
DELIMITER $$
CREATE OR REPLACE PROCEDURE TeeVaraus(IN p_asiakas_id INT, IN p_auto_id INT,
BEGIN
    DECLARE varauksien_maara INT;

    START TRANSACTION;

    SELECT COUNT(*) INTO varauksien_maara FROM Varaukset v
    WHERE v.auto_id = p_auto_id
    AND v.varaus_alkaa < p_varaus_paattyy
    AND v.varaus_paattyy > p_varaus_alkaa;

    IF varauksien_maara = 0 THEN

        INSERT INTO Varaukset(
            varaus_alkaa,
            varaus_paattyy,
            kokonaishinta,
            asiakas_id,
            auto_id

        ) VALUES(
            p_varaus_alkaa,
            p_varaus_paattyy,
            LaskeHinta(p_auto_id, p_varaus_alkaa, p_varaus_paattyy),
            p_asiakas_id,
            p_auto_id
        );

        UPDATE Autot
```

```

        SET Tila = 'vuokralla'
        WHERE auto_id = p_auto_id;
        COMMIT;

    ELSE

        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto on varattuna

    END IF;

END$$
DELIMITER ;

-- Audit-loki (AFTER UPDATE):
-- Luo triggeri Autot-tauluun. Aina kun auton tila muuttuu arvoon 'huollossa'
-- (esim. "Auto [rekisterinnumero] siirretty huoltoon").
DELIMITER $$
CREATE TRIGGER tr_au_autot_huollossa_loki_lisays
AFTER UPDATE ON Autot
FOR EACH ROW
BEGIN
    IF NEW.tila = 'huollossa' THEN
        INSERT INTO Loki(toimenpide, aikaleima, kenen_tekema)
        VALUES(CONCAT('Auto ', OLD.rekisterinnumero, ' siirretty huolt
    END IF;
END$$
DELIMITER ;

-- Turvatarkistus (BEFORE INSERT):
-- Luo triggeri Varaukset-tauluun, joka estää päällekkäiset varaukset tietoka
-- Jos uusi varaus osuu aikavälille, jolloin auto on jo varattu, triggerin tu
DELIMITER $$
CREATE TRIGGER tr_bi_tarkista_onko_auto_varattu
BEFORE INSERT ON Varaukset
FOR EACH ROW
BEGIN
    DECLARE varauksien_maara INT;

    SELECT COUNT(*) INTO varauksien_maara FROM Varaukset v
    WHERE v.auto_id = NEW.auto_id
    AND v.varaus_alkaa < NEW.varaus_paattyy
    AND v.varaus_paattyy > NEW.varaus_alkaa;

    IF varauksien_maara > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto on varattuna
    END IF;
END$$
DELIMITER ;

-- Näkymä (View): v_AktiivisetVaraukset
-- Luo näkymä, joka yhdistää tietoa tauluista: se näyttää asiakkaan nimen (mu
-- Näkymän tulee näyttää vain ne varaukset, jotka ovat tällä hetkellä käynnis
CREATE VIEW v_AktiivisetVaraukset AS

```

```
SELECT CONCAT(asiakas.etunimi, " ", asiakas.sukunimi), a.merkki, a.rekisterin  
JOIN autot a ON a.auto_id = v.auto_id  
JOIN asiakkaat asiakas ON asiakas.asiakas_id = v.asiakas_id  
WHERE v.varaus_paattyy > NOW();
```

```
-- Luo rooli asiakaspalvelu.
```

```
-- Anna tälle roolille oikeus lukea (SELECT) ja lisätä (INSERT) tietoa Varauk  
CREATE ROLE IF NOT EXISTS 'asiakaspalvelu';  
GRANT SELECT, INSERT ON Varaukset TO 'asiakaspalvelu';  
GRANT SELECT, INSERT ON Asiakkaat TO 'asiakaspalvelu';  
GRANT EXECUTE ON PROCEDURE TeeVaraus TO 'asiakaspalvelu';
```

```
-- Luo rooli mekaanikko. Anna tälle roolille oikeus päivittää (UPDATE) Autot-  
CREATE ROLE IF NOT EXISTS 'mekaanikko';  
GRANT UPDATE(tila) ON Autot TO 'mekaanikko';  
GRANT SELECT ON v_aktiivisetvaraukset TO 'mekaanikko';
```

```
-- Luo testikäyttäjät molemmille rooleille ja liitä roolit niihin (GRANT ROLE  
CREATE USER IF NOT EXISTS 'asiakaspalvelu'@'localhost' IDENTIFIED BY 'PASSWOR  
GRANT 'asiakaspalvelu' TO 'asiakaspalvelu'@'localhost';  
SET DEFAULT ROLE 'asiakaspalvelu' FOR 'asiakaspalvelu'@'localhost';
```

```
CREATE USER IF NOT EXISTS 'mekaanikko'@'localhost' IDENTIFIED BY 'PASSWORD';  
GRANT 'mekaanikko' TO 'mekaanikko'@'localhost';  
SET DEFAULT ROLE 'mekaanikko' FOR 'mekaanikko'@'localhost';
```

```
-- Kirjoita kysely, joka etsii tietyn asiakkaan kaikki varaukset. Aja kysely  
EXPLAIN SELECT v.asiakas_id, v.auto_id, v.varaus_alkaa, v.varaus_paattyy FRO  
JOIN Asiakkaat a ON v.asiakas_id = a.asiakas_id  
WHERE v.asiakas_id = 1;
```

```
-- Luo tarvittava indeksi (CREATE INDEX), joka nopeuttaa tätä kyselyä (esim.  
-- Aja EXPLAIN uudelleen ja vertaa tuloksia dokumentaatiossasi.  
CREATE INDEX idx_varaukset_asiakas_varaus_alkaa ON Varaukset (asiakas_id, var
```

```
-- Huom: Varmista ensin, että MariaDB:n tapahtuma-ajastin on päällä: SET GLOB  
SET GLOBAL event_scheduler = ON;
```

```
-- Luo tapahtuma PaivitaAutojenTilat, joka ajetaan automaattisesti kerran vuo  
-- Tapahtuman tulee etsiä varaukset, jotka ovat päättyneet eilen, ja päivittää  
DELIMITER $$
```

```
CREATE EVENT PaivitaAutojenTilat
```

```
ON SCHEDULE
```

```
EVERY 1 DAY
```

```
STARTS CURRENT_DATE + INTERVAL 23 HOUR + INTERVAL 59 MINUTE
```

```
COMMENT 'Päättyneiden varauksien autot on laitettu vapaiksi.'
```

```
DO
```

```
BEGIN
```

```
UPDATE Autot a
```

```
JOIN Varaukset v ON v.auto_id = a.auto_id
```

```
        SET a.tila = 'vapaa'
        WHERE v.varaus_paattyy <= NOW() - INTERVAL 1 DAY
        AND a.tila = 'vuokralla';

    END $$
DELIMITER ;

DELIMITER $$
CREATE EVENT VapautaKayttamatonVaraus
    ON SCHEDULE
        EVERY 1 DAY
        STARTS CURRENT_DATE + INTERVAL 23 HOUR + INTERVAL 59 MINUTE
    COMMENT 'Käyttämättömät varaukset ovat vapautettu.'
    DO
        BEGIN
            UPDATE Autot a
            JOIN Varaukset v ON v.auto_id = a.auto_id
            SET a.tila = 'vapaa'
            WHERE v.varaus_alkaa < NOW()
            AND a.tila = 'varattu';

        END $$
DELIMITER ;
```

Loppusanat

SQL on ihan kiva, mutta se osaa olla myös tosi turhauttava. Hyvin pienet kirjoitusvirheet tulevat heittämään suorituksen aikana virheen tai se ei aiheuta mitään virhettä vaan epäonnistuu äänettömästi.

Yksi esimerkki tästä on, että skriptissäni olin laittanut loki -taulun viimeisen tietorivin lisäyksen jälkeen ',', jonka jälkeen oli delimiteri. Skripti suoriutui alusta loppuun asti ja loi asiat niin kuin piti, mutta se olikin jättänyt Loki -taulun täysin tyhjäksi.

Testattavuuden "vaikeus" näkyy myös esim. noiden tapahtumien kanssa, ja ainoa kunnollinen tapa testata niiden toimintaa on luoda niille erillisiä tauluja ja asettaa se esim. laukeamaan 5 minuutin välein ja seuraamaan, mitä se saa aikaan.

Kurssipalaute

len käyttänyt SQL:llä, mutta vain pintakosketuksella; taulujen luonti, tietojen lisäykset/muuttamiset/poistot sekä muutamia Triggereitä. Näiden ulkopuolella, lähes kaikki oli uutta ja lähes aina mielenkiintoista.

Kurssissa oli juuri niitä asioita, joita olin myös täältä hakemassa, kuten proseduureista, triggereistä, käyttäjähallinnasta jne.. Asioita, joihin en ole juurikaan koskenut. Tämä todellakin laajensi ymmärrystäni asian suhteen.

Kurssi läksi hitaasti liikenteeseen ja hyvästä syystä, kun muutoksia oli kurssiin tapahtunut, mutta ennen puolta väliä oltiin jo päästy hyvään vauhtiin.

Yksi asia, mikä ei ollut itselle kiinnosta oli Power BI ja Excelin hyödyntäminen. On tosiaan ihan hyvä, että tätäkin demonstroidaan ja sen (tärkeää) olemassa oloa tuodaan esille, mutta ehkä tämäkin aika olisi ollut hyvä käyttää ihan SQL:än tutkimiseen; siellä on todella monia mielenkiintoisia asioita vielä tutkittavana. Toisena syynä voisi tähän skippaamiseen olla, että voisi kuvitella, että tämä olisi osana esim. liiketalouden puolta, jossa tämän tyyppisen datan kanssa saatetaan toimia enemmän?

Kaikinpuolin kurssi oli hyvä ja onnistunut, ja uskon, että tulevia vuosia varten se vielä vain paranee.

Kiitokset!